



WEEK 6 · CAPSTONE EXTENSION

การสร้างและปรับแต่ง Chatbot

ทฤษฎี UI/UX สำหรับ Chatbot และสถาปัตยกรรมโปรเจกต์ที่ปรับแต่งได้

หลักสูตร IT · KMITL



ภาพรวมเนื้อหาวันนี้

3 ส่วนที่เชื่อมกันเป็นเส้นเรื่องเดียว



PART 1

uv & Python Environment

- ทำไมต้องใช้ uv
- ติดตั้ง uv และ Python
- คำสั่งและไฟล์สำคัญ



PART 2

UI/UX สำหรับ Chatbot

- UX เทียบกับ UI
- องค์ประกอบของหน้าจอแชท
- จากทฤษฎีสู่โค้ดจริง



PART 3

สถาปัตยกรรม & การปรับแต่ง

- Config และตัวแปรสำคัญ
- เปลี่ยนโมเดล / ปลายทาง / prompt
- Data flow ทั้งระบบ



uv คืออะไร

- 01 **uv คือใคร** — เครื่องมือจัดการแพ็คเกจและโปรเจกต์ Python เขียนด้วยภาษา Rust โดยทีม Astral ผู้สร้าง Ruff
- 02 **รวมเครื่องมือหลายตัวไว้ในที่เดียว** — แทนที่ pip + venv + pyenv + pip-tools + poetry ด้วยคำสั่งเดียว
- 03 **จัดการได้ครบวงจร** — ติดตั้ง Python เอง สร้าง virtual environment อัตโนมัติ และล็อกเวอร์ชัน dependency ให้แน่นอน
- 04 **ใช้ uv เป็นเครื่องมือหลักในการจัดการโปรเจกต์นี้**



ทำไมต้องใช้ uv

- 01 **เร็วกว่าเดิมมาก** — เร็วกว่า pip แบบดั้งเดิมได้ถึง 10-100 เท่า เพราะเขียนด้วย Rust และ resolve dependency แบบขนาน
- 02 **All-in-one** — ไม่ต้องสลับใช้เครื่องมือหลายตัว ไม่ต้องจำคำสั่งของแต่ละเครื่องมือแยกกันอีกต่อไป
- 03 **ผลลัพธ์ตรงกันทุกเครื่อง** — ไฟล์ uv.lock รับประกันว่าทุกคนในทีมได้ dependency เวอร์ชันเดียวกันเป็น:
- 04 **มาตรฐานใหม่ของวงการ** — ชุมชน Python จำนวนมากย้ายมาใช้ uv แทนเครื่องมือเดิมตั้งแต่ปี 2024 เป็นต้นมา



ติดตั้ง uv

install-uv

Windows (PowerShell)

> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"

macOS / Linux (Terminal)

> curl -LsSf https://astral.sh/uv/install.sh | sh

ตรวจสอบว่าติดตั้งสำเร็จ

> uv --version



ติดตั้ง Python ผ่าน uv

- 01 `uv python install 3.14` — ติดตั้ง Python เวอร์ชันที่ต้องการ โดยไม่ต้องไปดาวน์โหลดจาก python.org เอง
- 02 `uv python list` — ดูรายการเวอร์ชัน Python ที่มีให้ติดตั้งหรือที่ติดตั้งไว้แล้วในเครื่อง
- 03 `uv python pin 3.14` — ล็อกเวอร์ชัน Python ให้กับโปรเจกต์นี้โดยเฉพาะ สร้างไฟล์ `.python-version` ให้อัตโนมัติ
- 04 **มาตรฐานเดียวกันทุกเครื่อง** — uv ดาวน์โหลด Python แบบ standalone build เก็บแยกจากระบบ ไม่ชนกับ Python ตัวอื่นที่มีอยู่แล้ว



คำสั่ง uv ที่ต้องรู้

คำสั่ง	หน้าที่
<code>uv init</code>	สร้างโปรเจกต์ Python ใหม่พร้อมไฟล์ตั้งต้นที่จำเป็น
<code>uv add <package></code>	เพิ่ม dependency เข้าโปรเจกต์ และอัปเดต pyproject.toml + uv.lock
<code>uv remove <package></code>	ลบ dependency ออกจากโปรเจกต์
<code>uv sync</code>	ติดตั้ง dependency ทั้งหมดให้ตรงกับที่ระบุไว้ใน uv.lock
<code>uv run <command></code>	รันคำสั่งภายใน virtual env โดยอัตโนมัติ ไม่ต้อง activate เอง



ไฟล์ที่ uv สร้างให้

- 01 `.python-version` — ระบุนเวอร์ชัน Python ที่ล็อกไว้สำหรับโปรเจกต์นี้โดยเฉพาะ
- 02 `pyproject.toml` — ระบุนชื่อโปรเจกต์และรายการ dependency ที่ "ต้องการ" ใช้
- 03 `uv.lock` — บันทึกเวอร์ชันที่ "resolve แล้วจริง" ของทุกไลบรารี รวมถึง dependency ของ dependency แบบละเอียด
- 04 `.venv/` — virtual environment ที่ uv สร้างและจัดการให้อัตโนมัติ

00

ความสัมพันธ์ pyproject.toml ↔ uv.lock

- 01 **pyproject.toml** บอกว่า “อยากได้อะไร” — เช่น `fastapi>=0.141.1` คือช่วงเวอร์ชันที่ยอมรับได้
- 02 **uv.lock** บอกว่า “ได้อะไรจริง” — เวอร์ชันตายตัวของทุกไลบรารีที่ผ่านการ resolve และทดสอบความเข้ากันได้แล้ว
- 03 ต้อง **commit** ทั้งสองไฟล์เข้า **git** เสมอ — เพื่อให้ทุกคนในทีมและทุกเครื่องได้ dependency ชุดเดียวกันเป๊ะด้วยคำสั่ง `uv sync`
- 04 ห้ามแก้ **uv.lock** ด้วยมือ — ปล่อยให้ uv เป็นคนจัดการไฟล์นี้ให้เท่านั้น



UX เทียบกับ UI

- 01 UI (User Interface)** — สิ่งที่เรา “มองเห็นและสัมผัสได้” — สี ปุ่ม ตัวอักษร ไอคอน ระยะห่างต่างๆ บนหน้าจอ
- 02 UX (User Experience)** — “ความรู้สึกและประสบการณ์โดยรวม” ที่ผู้ใช้ได้รับ ใช้งานง่ายไหม เข้าใจง่ายไหม
- 03 UI คือส่วนหนึ่งของ UX** — UI สวยไม่ได้แปลว่า UX ดีเสมอไป ต้องมองภาพรวมของการใช้งานทั้งหมด
- 04 ตัวอย่างใน Chatbot** — ปุ่มส่งข้อความสวยงาม (UI ดี) แต่ถ้ากดแล้วไม่รู้ว่ากำลังส่งอยู่หรือไม่ ก็ถือว่า UX แย่



Usability Heuristics

เลือก 4 ข้อจากหลัก Nielsen 10 Usability Heuristics ที่เกี่ยวข้องกับ Chatbot มากที่สุด

01 Visibility of System Status — ระบบต้องบอกผู้ใช้เสมอว่ากำลังเกิดอะไรขึ้น เช่น กำลังพิมพ์คำตอบอยู่

02 User Control & Freedom — ผู้ใช้ควรแก้ไขหรือยกเลิกการกระทำของตัวเองได้

03 Consistency & Standards — ใช้รูปแบบเดียวกันตลอดทั้งแอป ไม่ทำให้ผู้ใช้ต้องเรียนรู้ใหม่ทุกหน้า

04 Help Users Recover from Errors — ข้อความ error ต้องบอกสาเหตุและวิธีแก้ไขอย่างชัดเจน เข้าใจง่าย



หลักการพื้นฐานที่กำกับทุกองค์ประกอบ

- 01 Consistency (ความสม่ำเสมอ)** — ปุ่ม สี ระยะห่าง ต้องเหมือนกันทุกจุดในแอป
- 02 Feedback (การตอบสนอง)** — ทุกการกระทำของผู้ใช้ต้องมีการตอบกลับให้เห็นเสมอ
- 03 Affordance (การสื่อความหมายในตัวเอง)** — องค์ประกอบต้อง “ดูออก” ว่าใช้งานอย่างไร เช่น ปุ่มต้องดูเหมือนกดได้
- 04 Visual Hierarchy (ลำดับความสำคัญ)** — สิ่งที่สำคัญกว่าต้องเด่นกว่า ด้วยขนาด สี หรือตำแหน่ง



Anatomy ของหน้าจอ Chatbot

อ้างอิงจากโครงสร้างจริงใน templates/index.html (unที่ 10)





การออกแบบ Message Bubble

- 01 แยก user/AI ด้วยตำแหน่งและสี — `.message.user` ซิดขวา กับ `.message.ai` ซิดซ้าย ใช้สีต่างกันชัดเจน
- 02 Bubble-tail effect — border-radius ไม่เท่ากันทุกมุม เช่น border-bottom-right-radius: 4px ฝั่งผู้ใช้ ให้ดูเหมือนหางบับเบิล
- 03 รองรับ Markdown ในบับเบิล — ใช้ marked.js แปลง Markdown เป็น HTML แล้ว sanitize ด้วย DOMPurify ก่อนแสดงผลเพื่อความปลอดภัย



สถานะที่ Chatbot ต้องมี

System Status & Feedback

- 01 Loading / Typing Indicator** — จุดกระพริบ 3 จุด บอกผู้ใช้งานว่า AI กำลังประมวลผลอยู่
- 02 Error Banner** — แจ้งเตือนทันทีเมื่อเชื่อมต่อ LM Studio ไม่ได้ พร้อมบอกวิธีแก้เบื้องต้น
- 03 Disabled Input** — ล็อกช่องพิมพ์และปุ่มส่งระหว่างรอคำตอบ ป้องกันการส่งข้อความซ้ำซ้อน
- 04 Empty State** — ข้อความต้อนรับเมื่อเปิดหน้าแชทครั้งแรก ก่อนมีบทสนทนาใดๆ



สี & Typography สำหรับ Chatbot

- 01 **Flat Design** — ใช้สีทึบ ไม่มี gradient หรือเงาฟุ้ง เน้นความเรียบง่ายและอ่านง่าย สไตลที่องค์กรใหญ่แนว Gartner นิยมใช้
- 02 **Contrast ที่เพียงพอ** — สีตัวอักษรกับพื้นหลังต้องต่างกันชัดเจน เพื่อให้อ่านง่ายตามหลัก accessibility (WCAG)
- 03 **เลือก Font ให้เหมาะกับภาษา** — ฟอนต์ต้องรองรับภาษาไทยได้ชัดเจนอ่านง่าย เช่น Kanit ที่ใช้ในสไลด์นี้ทั้งหมด



Layout & Spacing สำหรับบทสนทนา

- 01 Whitespace ระหว่างข้อความ** — ระยะห่างที่เหมาะสมช่วยลดความอึดอัดสายตา อ่านสบายขึ้น
- 02 จัดกลุ่มข้อความ (Grouping)** — ข้อความจากฝั่งเดียวกันควรอยู่ใกล้กัน แยกจากฝั่งตรงข้ามให้ชัดเจน
- 03 Responsive / Mobile-first** — ใช้ @media query ปรับ max-width และขนาดตัวอักษรให้เหมาะกับหน้าจอมือถือ



Microinteraction ที่ทำให้ Chatbot รู้สึกมีชีวิต

- 01 Animation ของ Loading Dots** — ใช้ @keyframes bounce สร้างจังหวะขยับของจุดโหลด ให้รู้สึกระบบยัง “มีชีวิต”
- 02 Auto-scroll อัตโนมัติ** — เลื่อนไปข้อความล่าสุดทุกครั้งที่มีข้อความใหม่เข้ามา
- 03 Transition ของปุ่มส่ง** — เปลี่ยนสถานะ :disabled อย่างนุ่มนวลระหว่างรอคำตอบจาก AI



วิเคราะห์โค้ดจริง

Workshop สั้นๆ — เชื่อมทฤษฎีเข้ากับโค้ดที่เขียนไปแล้ว

- 01 **เปิดไฟล์ `templates/index.html`** — ซีจุดที่สอดคล้องกับ Anatomy ของหน้าจอ Chatbot ที่เพิ่งเรียน
- 02 **เปิดไฟล์ `static/css/style.css`** — หาโค้ดที่ตรงกับหลัก Flat Design, Contrast และ Bubble-tail effect
- 03 **เปิดไฟล์ `static/js/chat.js`** — สังเกตว่า Microinteraction ต่างๆ เช่น auto-scroll และ disabled state เขียนไว้ตรงไหน



ภาพรวมสถาปัตยกรรมโปรเจกต์





ไฟล์ config ทั้งหมด

- 01 `.env` — ค่าลับจริงของโปรเจกต์ เช่น URL, token, system prompt — ห้าม commit เข้า git
- 02 `.env.example` — เทมเพลตให้ทีมเห็นว่าต้องตั้งค่าตัวแปรอะไรบ้าง โดยไม่มีค่าจริงอยู่ในนั้น
- 03 `config.py` — คลาส Settings จาก pydantic-settings อ่านค่าจาก `.env` มาเป็น object ที่ type-safe ใช้ได้ทั่วทั้งแอป



ตัวแปรสำคัญที่ปรับแต่งได้

ตัวแปร	ความหมาย
LM_STUDIO_BASE_URL	ที่อยู่ปลายทางของ LM Studio server ที่จะเชื่อมต่อ
LM_STUDIO_MODEL	ชื่อ (key) ของโมเดลที่จะใช้ตอบคำถาม
LM_STUDIO_API_TOKEN	โทเคนยืนยันตัวตน ใช้เมื่อเปิด Require API Key ใน LM Studio
SYSTEM_PROMPT	คำสั่งกำกับพฤติกรรมของ AI ตั้งแต่ต้นบทสนทนา
APP_SESSION_SECRET	คีย์ลับสำหรับเซ็นชื่อ session cookie



วิธีเปลี่ยนโมเดล

- 01 **เปิด LM Studio แล้วโหลดโมเดลใหม่** — ไปที่แท็บ Local Server แล้วกด Start Server หลังโหลดโมเดลที่ต้องการเสร็จ
- 02 **ตรวจสอบ model key ที่ถูกต้อง** — เปิด `GET .../api/v1/models` เพื่อดูชื่อ key ของโมเดล
- 03 **แก้ไฟล์ .env** — ตั้งค่า `LM_STUDIO_MODEL` ให้ตรงกับ key ที่เจอ
- 04 **รีสตาร์ทเซิร์ฟเวอร์** — รันคำสั่ง `uv run uvicorn main:app --reload` ใหม่อีกครั้ง



วิธีเปลี่ยนปลายทาง (Endpoint)

- 01 **ค่าเริ่มต้น** — `LM_STUDIO_BASE_URL=http://127.0.0.1:1234` ชี้ไปที่เครื่องตัวเอง
- 02 **เปลี่ยนไปเครื่องอื่นในวง LAN** — เช่น `http://192.168.1.20:1234` หาก LM Studio เปิดให้เข้าถึงผ่านเครือข่ายได้
- 03 **ไม่ต้องแก้โค้ดเลย** — `config.py` ใช้ `pydantic-settings` อ่านค่าจาก `.env` โดยอัตโนมัติ แก้แค่ไฟล์ `.env` ก็พอ



วิธีปรับ System Prompt

- 01 **ฝังเข้า input ของ turn แรก** — ฟังก์ชัน `_build_input` ใน `lmstudio_client.py` ใส่ `SYSTEM_PROMPT` ไว้ในข้อความแรกของบทสนทนาเท่านั้น
- 02 **ปรับพฤติกรรม AI ได้ทั้งหมด** — กำหนด tone ภาษา ขอบเขตความรู้ และกฎการตอบ ผ่านคำนี้เพียงคำเดียว
- 03 **แนวทางเขียน Prompt ที่ดี** — ระบุบทบาทให้ชัดเจน + กำหนดกฎการตอบ + ระบุรูปแบบผลลัพธ์ที่ต้องการ



Data Flow ของ 1 ข้อความ

ตัวเลขวงกลมบนโครงสร้างไฟล์ = ขั้นตอนด้านซ้ายที่ไฟล์นั้นถูกใช้งาน

1 ผู้ใช้พิมพ์ข้อความ

ในหน้า templates/index.html แล้วกดปุ่มส่ง

2 chat.js ยิง request

เรียก POST /api/v1/chat/stream ผ่าน fetch API

3 routers/chat.py รับคำขอ

ตรวจสอบ/สร้าง session แล้วส่งต่อให้ service

4 services/lmstudio_client.py เชื่อมต่อ LM Studio

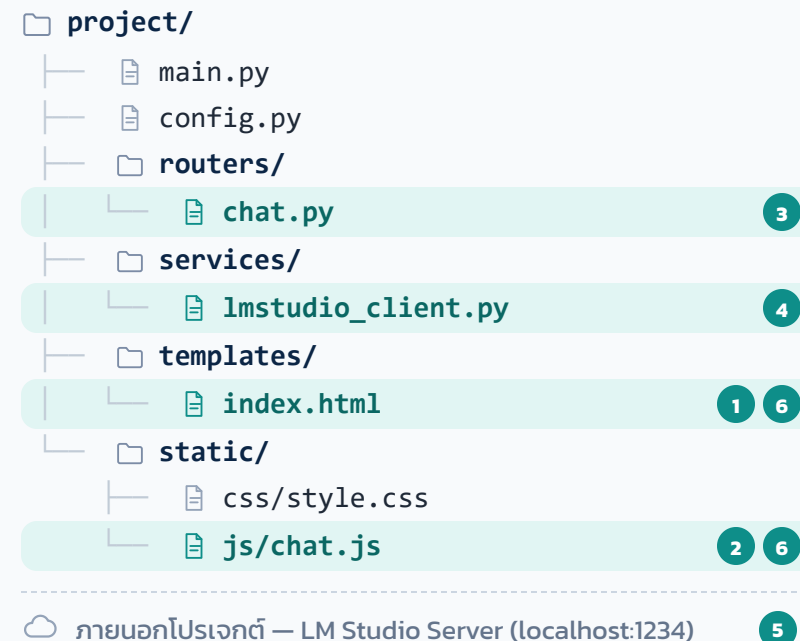
ส่ง payload พร้อม model และ input ไปยัง LM Studio server

5 LM Studio ส่งคำตอบกลับแบบ Stream

ทีละส่วนผ่าน Server-Sent Events (SSE)

6 chat.js render ผลลัพธ์แบบ real-time

แปลง Markdown และแสดงผลในบับเบิลกั้นที่ที่ได้รับแต่ละส่วน





Session ทำงานอย่างไร

- 01 คำขอแรก** — ไม่มี cookie มาด้วย เซิร์ฟเวอร์จึงสร้าง session_id ใหม่และเก็บลง conversation_store ในหน่วยความจำ
- 02 ส่ง cookie กลับไปที่ browser** — cookie ชื่อ session_id ถูกตั้งค่าแบบ httponly เพื่อความปลอดภัย
- 03 คำขอถัดไป** — browser ส่ง cookie กลับมาอัตโนมัติ เซิร์ฟเวอร์ใช้ previous_response_id ค่อยต่อเนื่องกับ LM Studio
- 04 ข้อจำกัดสำคัญ** — conversation_store เก็บในหน่วยความจำเท่านั้น รีสตาร์ทเซิร์ฟเวอร์แล้วทสทนทุกาทั้งหมดจะหายไป



แนวทางต่อยอด

- 01 **ปรับ Theme ของ UI** — เปลี่ยนสี ฟอนต์ หรือ layout ได้อิสระในไฟล์ `static/css/style.css`
- 02 **เพิ่มฟีเจอร์ใหม่** — เช่น อัปโหลดไฟล์ ส่งข้อความเสียง หรือสลับโหมด dark/light
- 03 **สลับ LLM Provider** — เปลี่ยนไปใช้ OpenAI, Ollama หรือผู้ให้บริการอื่น โดยแก้ไขเฉพาะไฟล์ `services/lmstudio_client.py`
- 04 **เก็บบทสนทนาถาวร** — ย้าย `conversation_store` จาก dict ในหน่วยความจำไปเป็น Redis หรือฐานข้อมูลจริง



สรุปภาพรวม

ผู้เรียนจะสามารถติดตั้ง ปรับแต่ง และอธิบายเหตุผลเบื้องหลัง Chatbot App ของตัวเองได้ทั้งหมด

- 01 **uv — เครื่องมือ** — ติดตั้งและจัดการโปรเจกต์ Python ได้ครบวงจรด้วยคำสั่งเดียว
- 02 **UI/UX — หน้าตา** — เข้าใจหลักการออกแบบที่ทำให้ Chatbot ใช้งานง่ายและน่าเชื่อถือ
- 03 **Architecture — โครงสร้างที่ปรับแต่งได้** — รู้ว่าต้องแก้ไฟล์ไหน ตัวแปรไหน เพื่อเปลี่ยนโมเดล ปลายทาง หรือพฤติกรรมของ AI



เตรียมของให้ครบก่อนเริ่ม

Pre-flight Checklist ก่อนย้ายโปรเจกต์ขึ้นเครื่อง/เซิร์ฟเวอร์จริง

- 01 ติดตั้ง uv ไว้แล้ว — ตรวจสอบด้วย `uv --version` ทวนจาก PART 1
- 02 ได้โค้ดโปรเจกต์มาอยู่ในเครื่องแล้ว — clone จาก git หรือคัดลอกทั้งโฟลเดอร์ project/ มาวางไว้บนเซิร์ฟเวอร์ปลายทาง
- 03 ติดตั้ง LM Studio และโหลดโมเดลไว้อย่างน้อย 1 ตัว — บนเครื่อง/เซิร์ฟเวอร์ที่จะรันจริง ไม่ใช่แค่เครื่องตอนพัฒนา
- 04 เช็กพอร์ตว่าง — 8000 สำหรับเว็บแอป และ 1234 สำหรับ LM Studio ต้องไม่ถูกโปรแกรมอื่นใช้อยู่ก่อน



ขั้นตอนที่ 1: ติดตั้ง Dependencies และตั้งค่า .env

install-project

1) เข้าไปที่โฟลเดอร์โปรเจกต์

> cd project

2) ติดตั้ง dependency ทั้งหมดด้วย uv (สร้าง .venv ให้อัตโนมัติ)

> uv sync

3) คัดลอกไฟล์ .env ตัวอย่าง แล้วค่อยแก้ค่าจริงในไฟล์ .env

> cp .env.example .env



ขั้นตอนที่ 2: เปิด LM Studio Server และตั้งชื่อโมเดล

- 01 **เปิดโปรแกรม LM Studio** — โหลดโมเดลที่ต้องการใช้งานให้เสร็จเรียบร้อยก่อน
- 02 **ไปที่แท็บ Local Server แล้วกด Start Server** — ค่าเริ่มต้นคือ `http://127.0.0.1:1234`
- 03 **เปิดเบราว์เซอร์ไปที่ปลายทาง `/api/v1/models`** — เพื่อดู "model key" ที่ถูกต้องของโมเดลที่โหลดไว้
- 04 **นำ key ที่ได้ไปใส่ในไฟล์ `.env`** — แกะค่า `LM_STUDIO_MODEL` ให้ตรงกับ key นั้นเป๊ะ



ขั้นตอนสุดท้าย: รันเซิร์ฟเวอร์ด้วย `uv run`

run-server

รันเว็บแอปผ่าน uv โดยตรง ไม่ต้อง activate .venv เอง

> uv run uvicorn main:app --reload

แล้วเปิดเบราว์เซอร์ไปที่

`http://localhost:8000`

`--reload` คือโหมดสำหรับพัฒนา (dev only) รีสตาาร์ทอัตโนมัติทุกครั้งที่แก้โค้ด



เช็คให้ชัวร์ว่าใช้งานได้จริง

ผ่านครบทุกข้อ = โปรเจกต์พร้อมใช้งานจริงแล้ว

- 01 **เห็นหน้าแชทพร้อมข้อความต้อนรับ (Empty State)** — แปลว่า static files และ template ถูกเสิร์ฟถูกต้อง
- 02 **พิมพ์ข้อความแล้วได้คำตอบกลับมาแบบ Stream ทีละส่วน** — แปลว่าเชื่อมต่อ LM Studio และ SSE ทำงานครบวงจร
- 03 **ปุ่ม “หยุด” และ “เริ่มใหม่” ทำงานตามที่ควร** — ทวนจาก PART 2 ที่เพิ่งปรับให้ตรงกับทฤษฎี User Control & Freedom
- 04 **ลองปิด LM Studio แล้วส่งข้อความ ต้องเห็น Error Banner ทันที** — ยืนยันว่า error handling จากบทที่ 12 ยังทำงานถูกต้อง